Introduction to GDB & Valgrind

Presenter: Irtaza Hyder

Introduction

What is Debugging?

debugging (noun)

the process of identifying and removing errors from computer hardware or software

- Oxford Dictionary

debugging (noun)

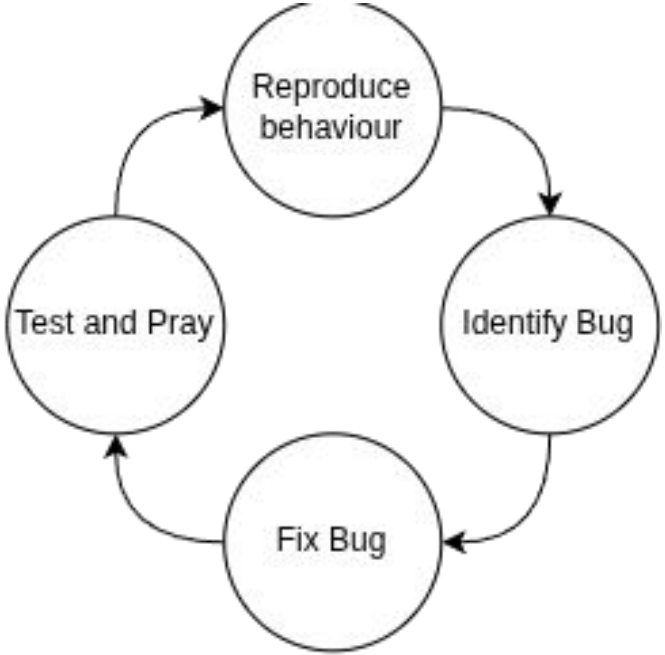
an emergent activity that arises when the mental model of a program or its data differs from its observed runtime behavior, and the underlying reason cannot be easily found

- Dawid Zalewski

Debugging Loop

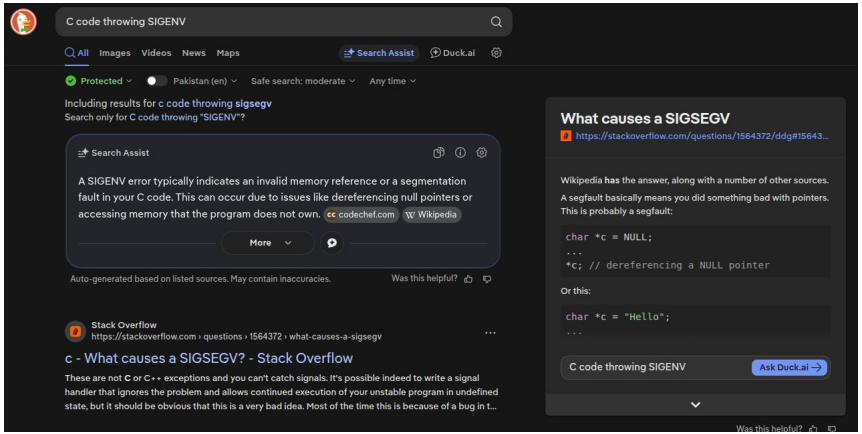
Resources:

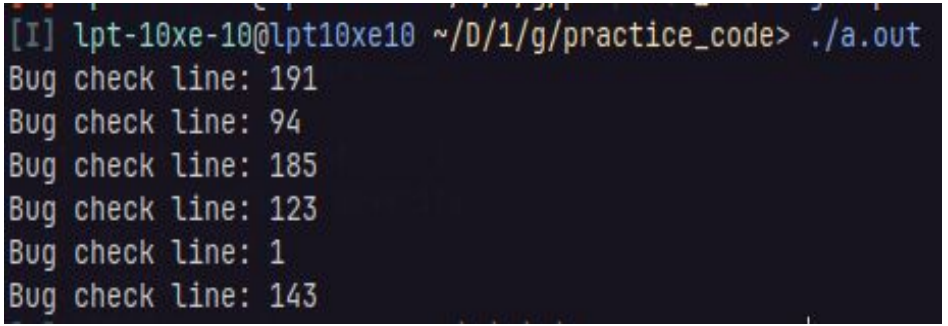
1. Logs
2. Static Analyzers
3. Dynamic Analyzers
4. Profiler



What you might be doing

1. Print <Everywhere>
2. Google Error message
3. Ask AI
4. Ask a Friend





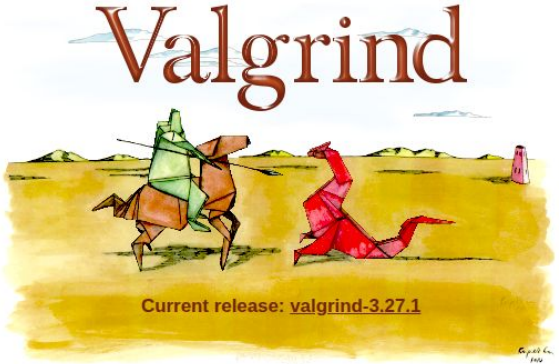
What we would be learning

1. Understand Error messages
2. Divide and Conquer
3. **Use debugging tools**

Tools

GDB: https://sourceware.org/gdb/

Valgrind: https://valgrind.org/



Getting Started

Compiler Flags (GCC)

bash | enabling debug info

```
gcc -O0 -g <sources> -o <target>
```

-O0: Disable optimization **-g** : Emit debugging information in OS's native format

bash | enabling debug info

```
gcc -Og -g <sources> -o <target>
```

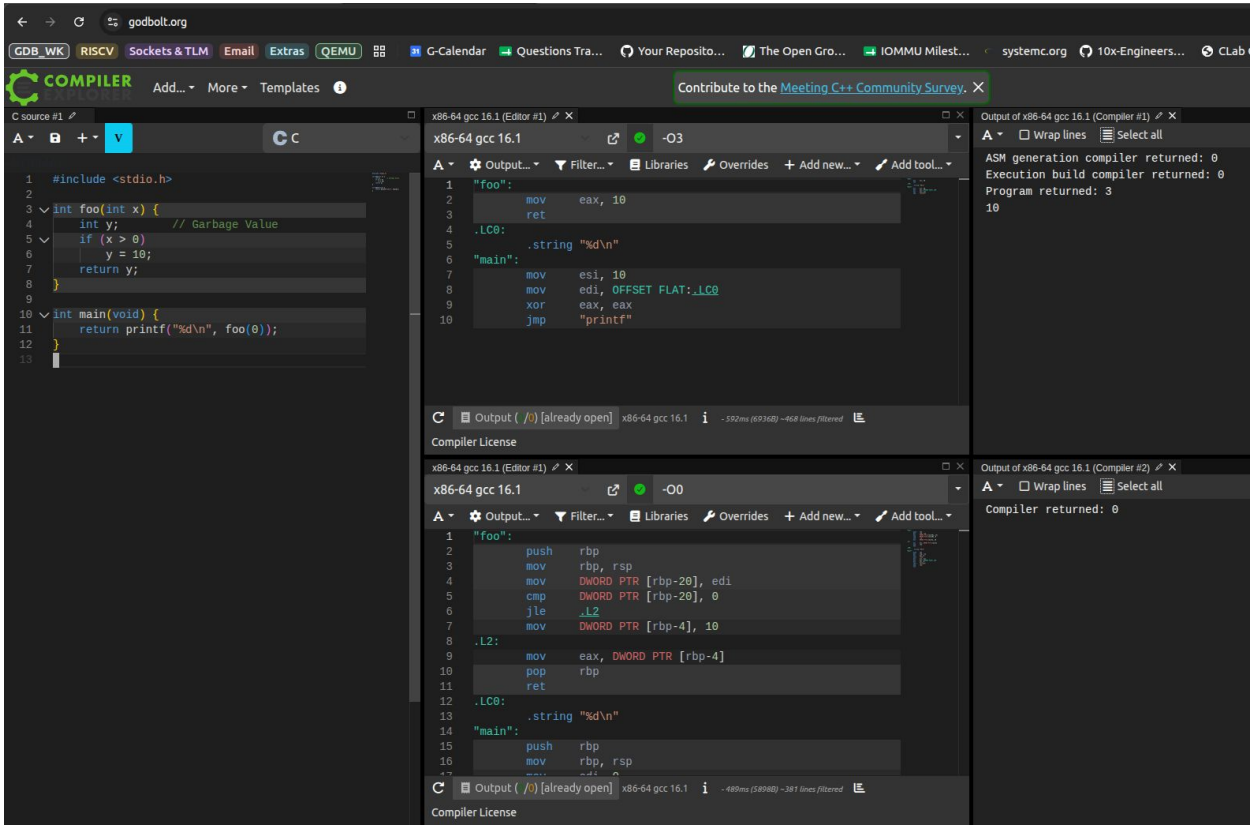
-Og: Optimize program for debugging

bash | enabling debug info

```
gcc -Og -ggdb <sources> -o <target>
```

-ggdb: Emit debug symbols in format best for gdb

Issues with -O0



Compiling programs

For your ease a run the following command to generate the output programs:

bash | cmake

```
cmake --preset host

cmake --build build (RUN THIS EVERYTIME YOU CHANGE ANY FILE)
```

This generates all the executable programs in build/<dir>/<executable>.

gdb_basics/task1.c ⇒ build/gdb_basics/task1

Running GDB

```
[I] lpt-10xe-10@lpt10xe10 ~/D/1/g/practice_code> gdb --version
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
[I] lpt-10xe-10@lpt10xe10 ~/D/1/g/practice_code> gdb
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
```

Basics of GDB

GDB Basics

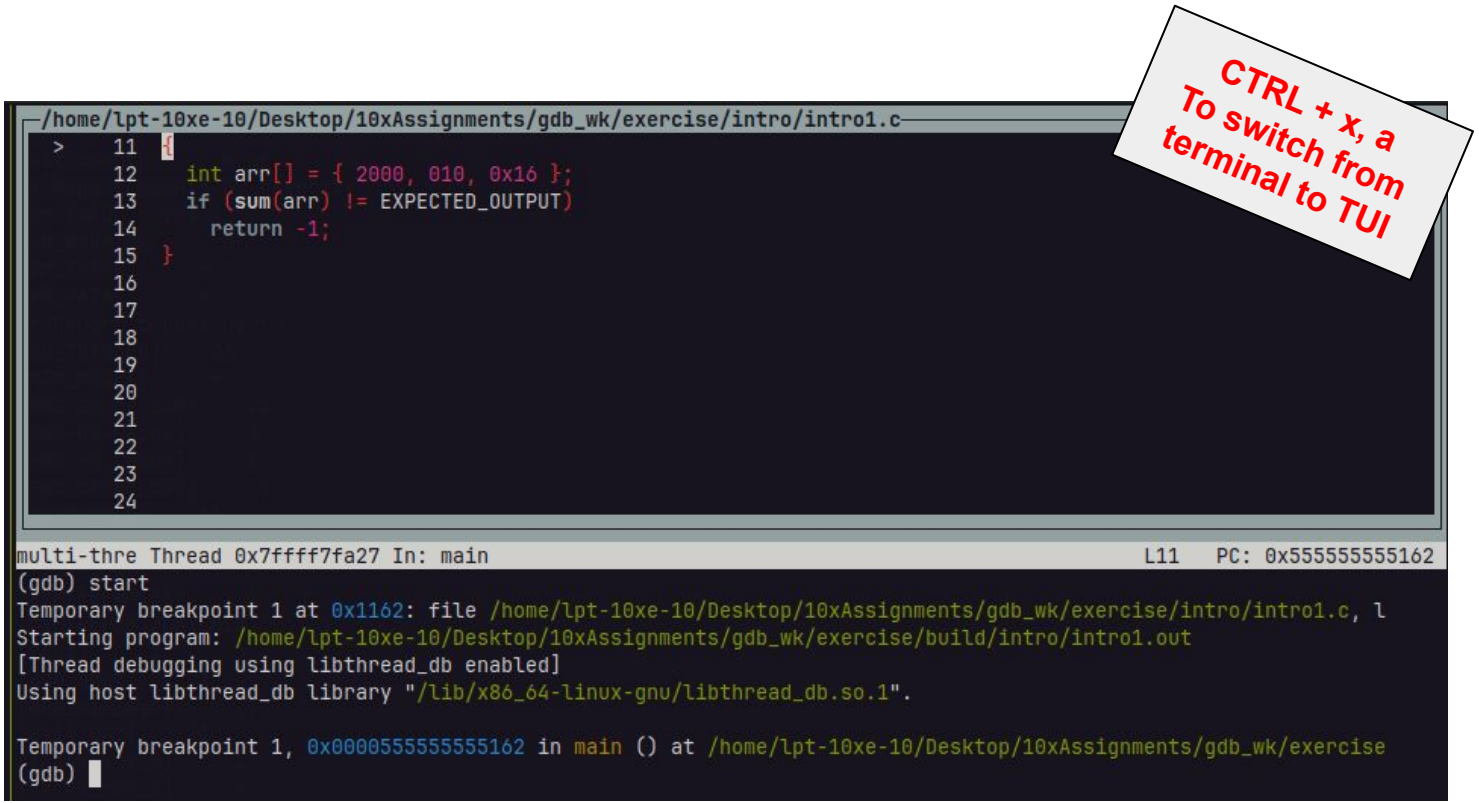
bash | gdb basics

gdb build/gdb_basics/task1

help <cmd>	Displays information about @cmd usage in GDB
run / r	Executes program in GDB
start	Starts executing program and stops at entry point (main)
next / n [N]	Continues execution till next @N source line
print / p <expr>	Evaluates expression and prints result
list / l [N]	Lists the source code. If line given, lists ±10 lines from line @N
quit / q	Exit GDB


(gdb) start
Temporary breakpoint 1 at 0x1162: file /home/lpt-10xe
Starting program: /home/lpt-10xe-10/Desktop/10xAssign
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gn
Temporary breakpoint 1, 0x0000555555555162 in main ()
9 {
(gdb) list
4 {
5 return arr[0] + arr[1] + arr[2];
6 }
7
8 int main()
9 {
10 const int expected_output = 2026;
11 int arr[] = { 2000, 010, 0x16 };
12 if (sum(arr) != expected_output)
13 return -1;
(gdb) n
11 int arr[] = { 2000, 010, 0x16 };
(gdb) p sum(arr)
$1 = 4096
(gdb)

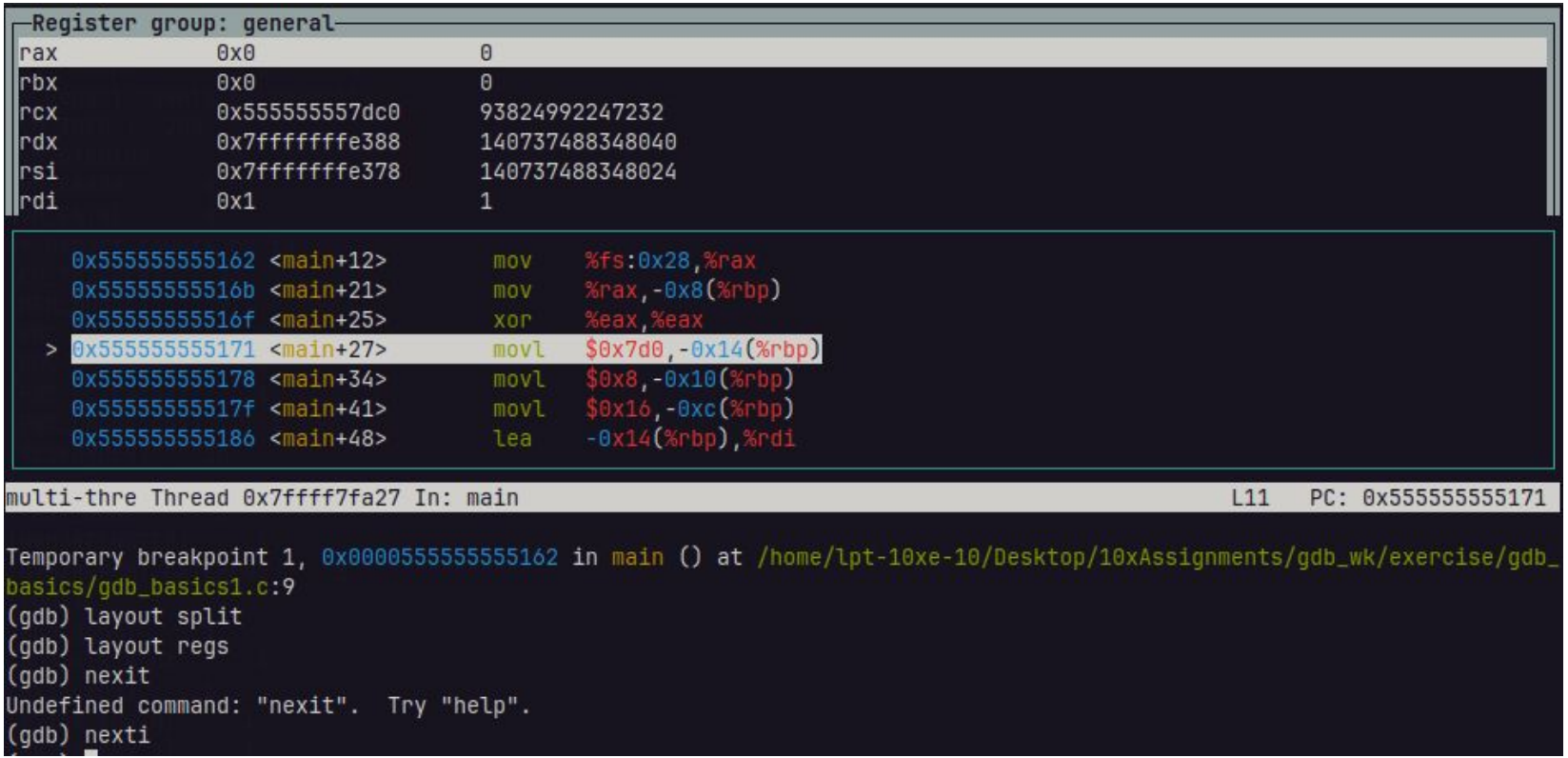
TUI


CTRL + x, a
To switch from
terminal to TUI
/home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise/intro/intro1.c
11 {
12 int arr[] = { 2000, 010, 0x16 };
13 if (sum(arr) != EXPECTED_OUTPUT)
14 return -1;
15 }
multi-thre Thread 0x7ffff7fa27 In: main
L11 PC: 0x555555555162
(gdb) start
Temporary breakpoint 1 at 0x1162: file /home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise/intro/intro1.c, l
Starting program: /home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise/build/intro/intro1.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Temporary breakpoint 1, 0x0000555555555162 in main () at /home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise
(gdb)

Commands

Ctrl + x, a	Switch between GDB terminal and TUI
Ctrl + l	Refreshes TUI
layout	Select TUI layouts. Layouts: **src, reg, asm**
layout split	Splits current display screen
info registers	Shows the contents of registers
disassemble	Displays assembly instructions
nexti /ni [N]	Execute N assembly instructions

TUI Layout


Register group: general
rax 0x0 0
rbx 0x0 0
rcx 0x555555557dc0 93824992247232
rdx 0x7fffffffe388 140737488348040
rsi 0x7fffffffe378 140737488348024
rdi 0x1 1
0x555555555162 <main+12> mov %fs:0x28,%rax
0x55555555516b <main+21> mov %rax,-0x8(%rbp)
0x55555555516f <main+25> xor %eax,%eax
> 0x555555555171 <main+27> movl $0x7d0,-0x14(%rbp)
0x555555555178 <main+34> movl $0x8,-0x10(%rbp)
0x55555555517f <main+41> movl $0x16,-0xc(%rbp)
0x555555555186 <main+48> lea -0x14(%rbp),%rdi
multi-thre Thread 0x7ffff7fa27 In: main L11 PC: 0x555555555171
Temporary breakpoint 1, 0x0000555555555162 in main () at /home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise/gdb_basics/gdb_basics1.c:9
(gdb) layout split
(gdb) layout regs
(gdb) nexit
Undefined command: "nexit". Try "help".
(gdb) nexti

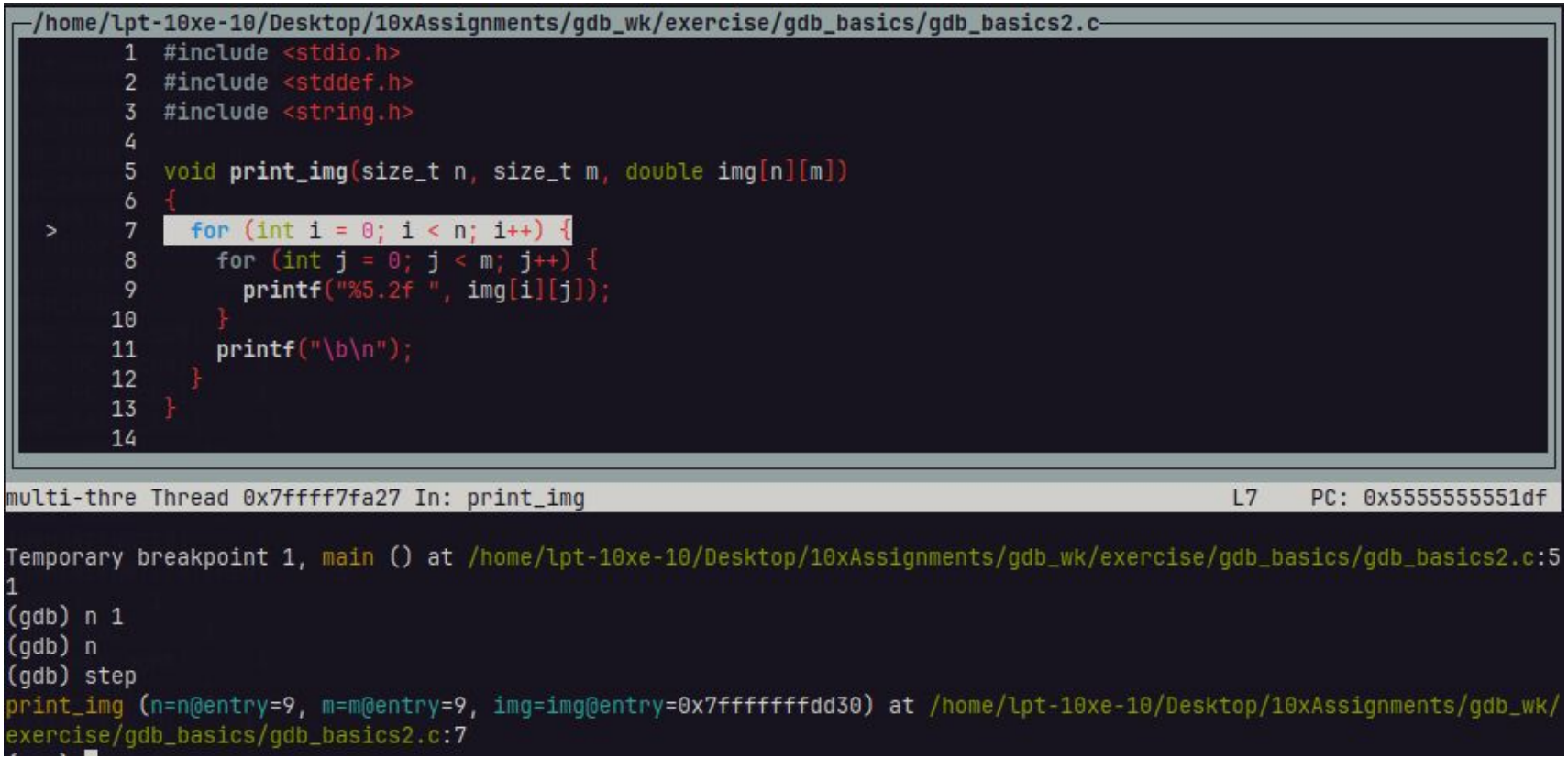
GDB Basics

bash | gdb basics

gdb build/gdb_basics/task2

step / s	Step inside function
finish / fin	Exit out of function
x[FMT] ADDRESS	Displays content of @ADDRESS based on given @FMT. FMT: (N)(format)(size) i.e. x/9xw 0x800000
continue / c	Continue program execution till next breakpoint.
until / u [loc]	Executes code until @loc. Useful when trying to exit loops

Step and Finish



Displaying memory elements

/home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/exercise/gdb_basics/gdb_basics2.c

```
1  #include <stdio.h>
2  #include <stddef.h>
3  #include <string.h>
4
5  void print_img(size_t n, size_t m, double img[n][m])
6  {
7    for (int i = 0; i < n; i++) {
8      for (int j = 0; j < m; j++) {
9        printf("%5.2f ", img[i][j]);
10     }
11     printf("\b\n");
12   }
13 }
14
```

multi-thre Thread 0x7ffff7fa27 In: print_img L7 PC: 0x5555555551df

```
(gdb) s
print_img (n=n@entry=9, m=m@entry=9, img=img@entry=0x7fffffffdd30) at /home/lpt-10xe-10/Desktop/10xAssignments/gdb_wk/
exercise/gdb_basics/gdb_basics2.c:7
(gdb) p img
$1 = (double (*)[variable length]) 0x7fffffffdd30
(gdb) x/3fg img
0x7fffffffdd30: 20      10
0x7fffffffdd40: 20
```

Breakpoints

Location specification

GDB’s way of defining location in code. Users can define code locations via:

1. Source Line: [*main.c:]50*
2. Offset: *-5*
3. Function name: *[foo.c:]foo*
4. Label

Breakpoints

Breakpoints halts program execution at the specified location.

break <lockspec>	Adds breakpoint to specified location
info breakpoints	Displays information about all current breakpoints.
delete breakpoints [b]	Deletes all given breakpoints. If none given, deletes all breakpoints

Prints

Want to add prints? GDB's got you!

display/[FMT] [address/expr]	Whenever program halts, GDB prints the value of the display variables
info display	Shows active displays
undisp [#num]	Deletes the display @num
dprintf <locspec> <fmt> <args>	Mix of breakpoint and printf. Gives the same effect of adding printf to your code.
info locals	Displays value of local variables in current scope.
info args	Displays value of function argument.

Watchpoints

Watchpoints are useful when we want to halt execution whenever variables are modified.

watch <expr>	Breakpoint halting execution of program whenever variable is modified.
info watchpoints	Displays information about all watchpoints.

NOTE: Watchpoint is a type of breakpoint. Hence `info breakpoints` and `delete breakpoints` apply to it as well.

Backtrace

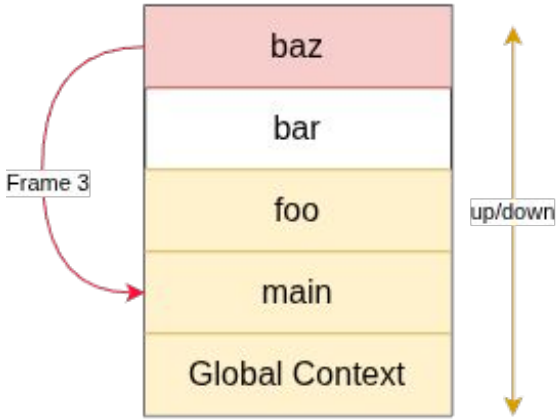
The user can view the callstack/function record at any location.

```
void baz ()
{
  // do something
}

void bar()
{
  // do something
  baz();
}

void foo()
{
  bar();
  // do something
}

int main()
{
  foo();
}
```

```
Breakpoint 1, baz () at test.c:2
2       {
(gdb) bt
#0  baz () at test.c:2
#1  0x000055555555514b in bar () at test.c:11
#2  0x0000555555555165 in foo () at test.c:17
#3  0x0000555555555173 in main () at test.c:23
```

Backtrace

backtrace or bt	Prints the callstack
bt full	Same as bt but with more details
frame / f [num]	Goes to frame #num
up [count]	Goes #count frames up
down [count]	Goes #count frames down



Traversing functions

call <expr>	Same as print but also applicable to call void functions.
finish	Continues execution till end of function
ret <expr>	Returns value of expression
skip <function>/<file>	When using step, skips going into @function or functions listed in @file
info skip	Displays details about the specified skips

Saving and Loading breakpoints

save breakpoints <filename>	Saves all current breakpoints to @filename
source breakpoints <filename>	Loads breakpoint information from filename

Task 3

bash | breakpoints

```
gdb build/breakpoints/task3
```

1. Malloc updates the original array's pointer. Because of this array **fib is never updated**.
2. **Stack smashing** generating SIGABRT because stack value becomes corrupted. For this task, stack is corrupted because of writing an 11th entry in fib. **Line 19 (should be i < n).**

Task 4

bash | breakpoints

gdb build/breakpoints/task4

1. **Stack smashing due to line 36** (should be i % 2)
2. **No error handling** for negative inputs from user
3. **sum_fib argument is char instead of int**, causing overflow large entries (i.e. 130)
4. **Realloc needs n * size(int), not just n**.

Coredumps

A core file or core dump is a file that **records the memory image of a running process and its process status** (register values etc.). Its primary use is post-mortem debugging of a **program that crashed while it ran outside a debugger**.

bash | Enabling coredump

```
ulimit -c unlimited

gdb <core-file> core
```

Task 5 (Debugging with core file)

bash | breakpoints

```
gdb ./build/breakpoints/task5

(gdb) core <core_file>
```

1. **bank.c:compare_username** comparison is incomplete (i.e. does not check full strings).
2. **bank.c:new_user function** performs pre-increment instead of post-increment. This is why we obtain wrong totals
3. **bank.c:account_info** contains **wild pointer** and still prints out value even when the user is not found!

Valgrind

What is Valgrind

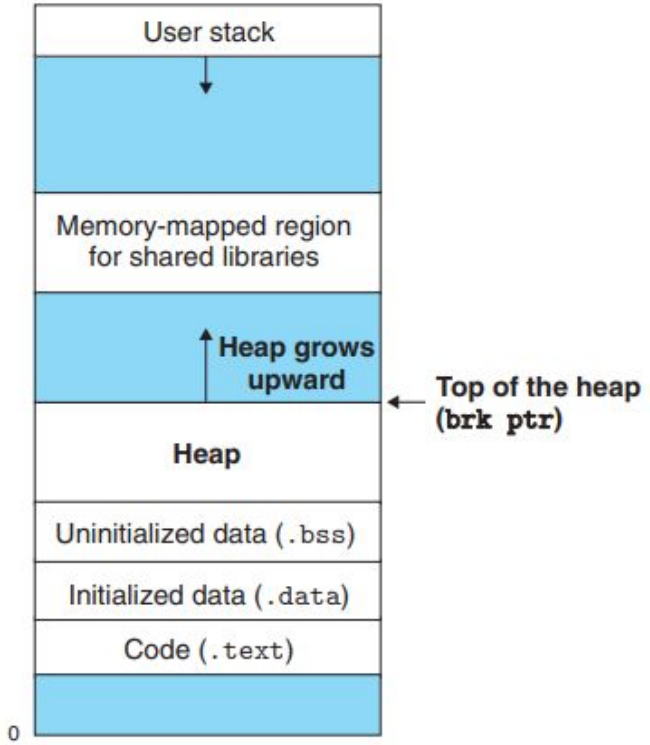
Where GDB lets you view in great detail what your program is doing, Valgrind attempts to analyze the behavior of your program and tell you where your program is doing things it shouldn’t.

We will mainly be focusing on Memcheck utility.

Stack and Heap

Stack: Compile time memory allocated to program. GDB displays the stack using the `backtrace` command.

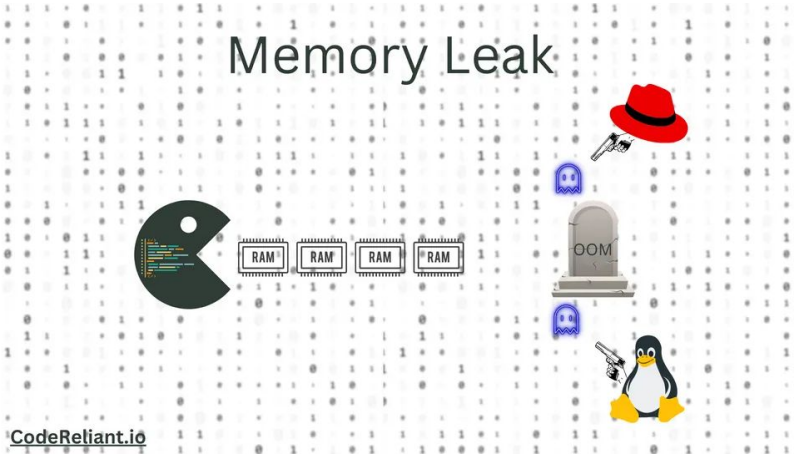
Heap: Memory pool allocated by the OS on demand. We will use Valgrind to observe objects in the heap.



Memory Leak

Unintentional memory consumption by a computer program where the **program fails to release memory** that is no longer needed or used.

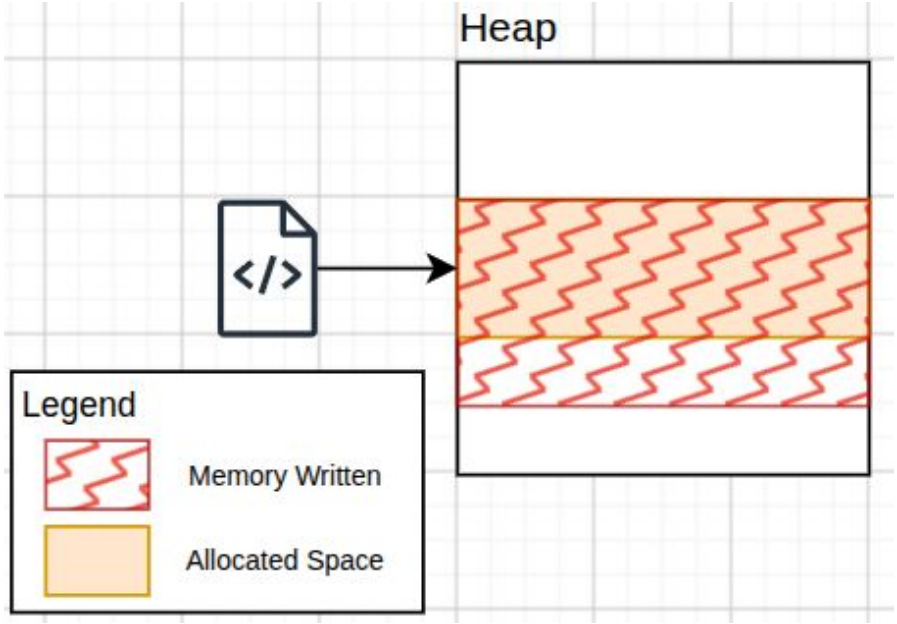
For languages like C and C++, often times the programmers is responsible to free any allocated memory.



Buffer Overrun

When a program **writes more data to a memory buffer than it can hold**, causing the extra data to **overwrite adjacent memory and potentially corrupt data or executable code**.

Stack smashing is a type of buffer overrun.



Running Valgrind

bash | valgrind

```
valgrind -v --leak-check=full --track-origins=yes [--tool=memcheck] ./<executable> <executable_args>
```

-v: Run valgrind in verbose mode. Prints warning about “unusual program behavior”.

--leak-check: Search for memory leaks in program.

-track-origins: Search for uninitialized memory used in dangerous way.

--tool=memcheck: Use the memcheck tool. This is the default option so can be left blank

Valgrind Task 1

bash | build/valgrind

```
valgrind --leak-check=full --track-origins=yes ./valgrind_task1 Hi there, hello world!
```

strdup function calls malloc to generate a string. node->data is also allocated on the heap and needs to be freed.

In task1.c:free_list free node->data.

Valgrind Task 2

bash | build/valgrind

```
valgrind --leak-check=full --track-origins=yes ./valgrind_task2 Hi there, hello world!
```

We are checking for the next node after freeing the current node. After delete_node() is called, list is now a **dangling pointer**.

1. Remove the Task2 free_list implementation and use Task1's free_list function.
2. The test would still fail due to the final dangling pointer (linked_list) in print_args. Set it to NULL after free_list is called (inside the ifdef block).

Valgrind Task 3

bash | build/valgrind

valgrind --leak-check=full --track-origins=yes ./valgrind_task3

1. For inputs greater than 11 bytes, the program causes buffer overrun and overwrites the string null terminator, which causes printf to access memory beyond what was allowed.
2. For shorter strings printf works as expected, but our byte read function tells read() to take 13 bytes in input, but our string container is 11 bytes (excluding null character).

Update **valgrind_task3:16** => read(..., string_length - 1);

Valgrind Task 4

bash | build/valgrind

```
valgrind --leak-check=full --track-origins=yes ./valgrind_task4
```

1. Improper use of strncpy. Update **valgrind_task4:19** to use memmove instead. (See `man strncpy`).
2. Buffer overrun can occur if offset is larger than string size. Add a check to ensure user given offset < string_length.

Valgrind Task 5

bash | build/valgrind

```
valgrind --leak-check=full --track-origin=yes ./valgrind_task5
```

1. Valgrind warns us about uninitialized memory on the stack being used in a dangerous way.
2. Another failure is due to freeing memory not allocated. This is due to **pre-increment in valgrind_task5.c:12**.

Spike with GDB

Prerequisites (RV64 GNU Toolchain)

1. RISCV GCC Toolchain: https://github.com/riscv-collab/riscv-gnu-toolchain
   a. Download the riscv32-elf-ubuntu-<*22.04/24.04*>-gcc.tar.xz from: https://github.com/riscv-collab/riscv-gnu-toolchain/releases?page=1. The installation would be based on the Ubuntu version you are using.
   b. Install the required dependencies:
      `$ sudo apt-get install autoconf automake autotools-dev curl python3 python3-pip python3-tomli libmpc-dev libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool patchutils bc zlib1g-dev libexpat-dev ninja-build git cmake libglib2.0-dev libslirp-dev libncurses-dev`
   c. Unzip the tar file. Use the unzipped folder's name in the next steps.
2. Add the following variable to your .bashrc:
   a. export RISCV="<path_to_riscv_gnu_toolchain>"
   b. export PATH="$RISCV:$PATH"
   c. $ source .bashrc
   d. $ riscv64-unknown-elf-gdb --version

Prerequisites (OpenOCD)

1. Ensure the following dependencies are met:
    a. make
    b. libtool
    c. pkg-config >= 0.23 or pkgconf
    d. autoconf >= 2.69
    e. automake >= 1.14
    f. texinfo >= 5.0

Prerequisites (OpenOCD) Cont…

2. Run the following commands:

bash

```
git clone git://git.code.sf.net/p/openocd/code openocd
git checkout v0.12.0
git merge 400cf2        # Contains fixes to some gnu issues
./bootstrap
./configure --enable-remote-bitbang --disable-werror
make -j $(nproc)
make install
```

How to run with OpenOCD (Revision)

Execute the steps in the exact order given below. Compilation can be skipped, the elf files are given for all the spike tasks for user's ease.

bash | compiling | term1

```
riscv64-unknown-elf-gcc -g -Og --specs=semihost.specs -o rot13 rot13.c # Optional

spike --rbb-port=9824 -m0x10000:0x20000 --halted spike.build/spike/rot13
```

bash | openocd | term2

```
openocd -f spike.cfg
```

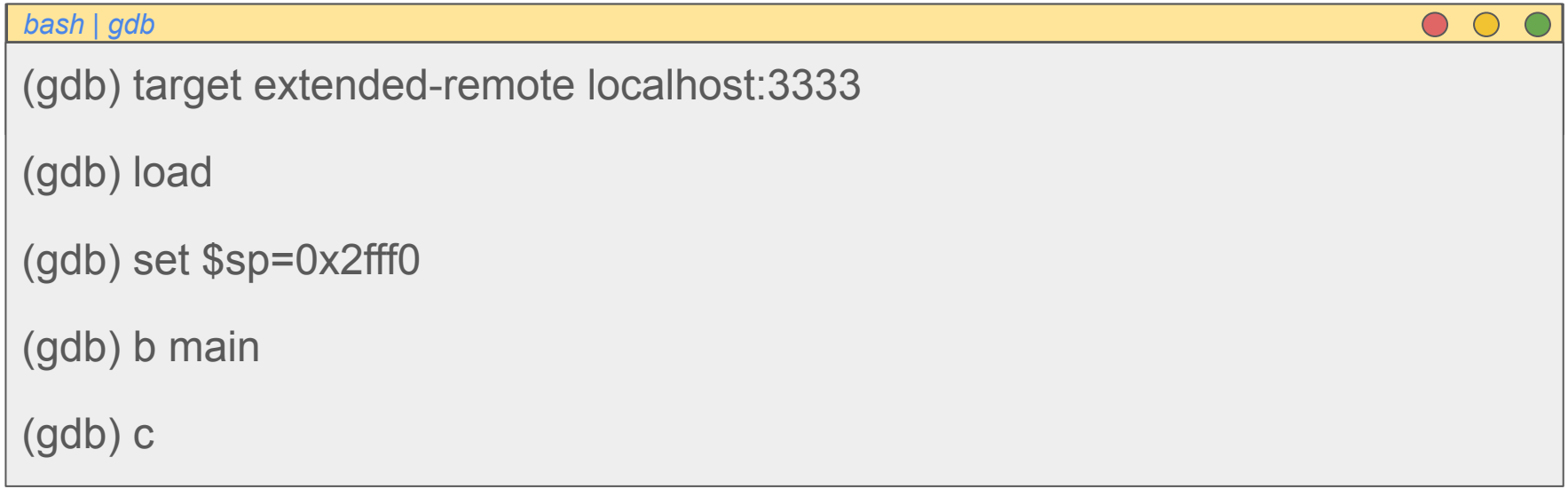
bash | GDB | term3

```
riscv64-unknown-elf-gdb rot13
```

OpenOCD

```
[I] lpt-10xe-10@lpt10xe10 ~/D/1/g/exercise (main)> openocd -f spike.cfg
Open On-Chip Debugger 0.12.0+dev-01611-g400cf213c-dirty (2026-08-02-19:44)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.org/doc/doxygen/bugs.html
Info : auto-selecting first available session transport "jtag". To override use 'transport select <transport>'.
Info : Initializing remote_bitbang driver
Info : Connecting to localhost:9824
Info : remote_bitbang driver initialized
Info : Note: The adapter "remote_bitbang" doesn't support configurable speed
Info : JTAG tap: riscv.cpu tap/device found: 0xdeadbeef (mfg: 0x777 (Fabric of Truth Inc), part: 0xeadb, ver: 0xd)
Info : datacount=2 progbufsize=2
Info : Examined RISC-V core; found 1 harts
Info :  hart 0: XLEN=64, misa=0x800000000014112d
Info : [riscv.cpu] Examination succeed
Info : starting gdb server for riscv.cpu on 3333
Info : Listening on port 3333 for gdb connections
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
```

Running GDB (Revision)



Bubble Sort

bash | breakpoints

riscv-elf-unknown-gdb spike/bubble_sort/bubble_sort

1. Checker does not decrement a1 by 1, i.e. check goes from range [0:6] which are 7 elements instead of 6.
2. Bubble sort skips the last element when sorting. Update bubble_sort.S:34 to li a0, 0.

Advanced Features in GDB

Breakpoint Features

We can configure breakpoint to trigger on certain conditions:

disable [b]	Diables breakpoint @b. Breakpoint @b would have no effect on program.
enable [b]	Enables breakpoint @b
enable once [b]	Breakpoint @b is disabled after it is hit once
enable count <N> <b>	Breakpoint @b is disabled after it is hit @N times.
condition <b> <expr>	Breakpoint @b is triggered only when @expr is true.
condition <b>	Removes condition from breakpoint @b
ignore <b> <count>	Set an ignore count = @N on @b. Breakpoint remains disabled while ignore count > 0.

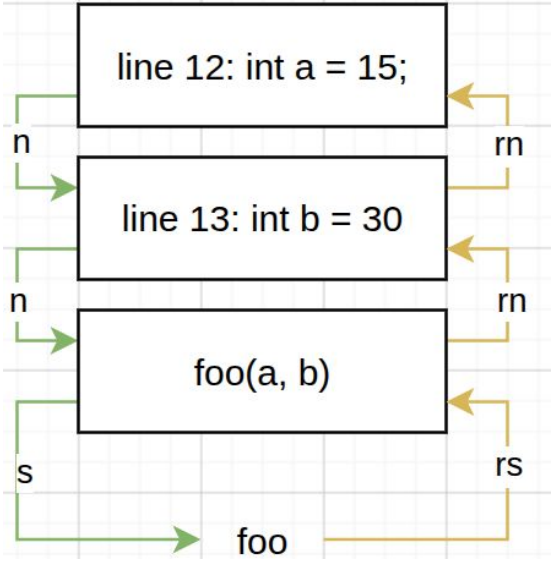
Set

Setting variables to be used in a GDB session. $<var> are called convenience variables which can be used throughout the GDB session.

Set var <var> = <value>	Sets @var in current scope to value <value>
p <var> = <value>	Same as using set var
set $<reg> = <value>	Sets value of register @reg to <value>
set args <value>	Sets value of function arguments to given list of <value>
set $<var> = <value>	Sets a convenience variable with value <value>
*array@len	Displays @len content of array.

Reverse Debugging

record [all]	Starts recording program execution. This is required to run reverse debugging commands.
reverse-next / rn	Same as next but program executes backwards.
reverse-step / rs	Same as step but program executes backwards.
reverse-continue / rc	Same as continue but program executes backwards.
record save <file>	Save execution log in @file.
record restore <file>	Loads execution log from @file.
record stop / s	Stop the record/replay target



Checkpoints (Linux only)

Saves snapshot of program's state. Returning to a checkpoint effectively undoes everything that has happened in the program since the checkpoint was saved.

checkpoint	Save a snapshot of the debugged program's current execution state.
info checkpoints	Lists saved checkpoints. Contains: Checkpoint ID, Process ID, Code Address and source-line/labels
restart <checkpoint-id>	Restore the program state that was saved as checkpoint @checkpoint-id.
delete checkpoint <checkpoint-id>	Delete the previously-saved checkpoint @checkpoint-id.

Jump

jump / j <locspec>:

Continue debugging program at a location of our choosing (similar to goto).

Attaching to a running Process

Sometimes we want to debug a running process. I.e. we want to inspect how Spike might be running under the hood.

bash | terminal 2 | spike

```
spike pk spike.build/spike/rot13
```

bash | terminal 1 | gdb

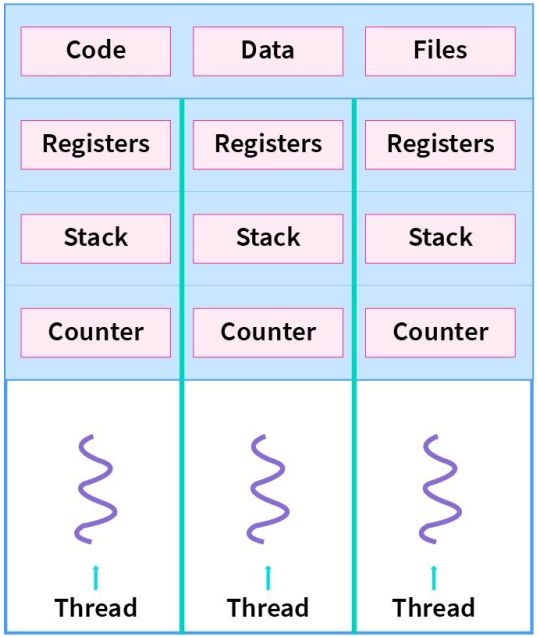
```
sudo gdb -p $(pidof spike)
```

Spike internals

```
[N] lpt-10xe-10@lpt10xe10 ~/D/1/g/m/debug_me_live (main)> sudo gdb -p $(pidof spike)
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Attaching to process 1762374
Reading symbols from /home/lpt-10xe-10/Desktop/10xAssignments/RISCV64/riscv-isa-sim/build/spike...
Reading symbols from /lib/x86_64-linux-gnu/libstdc++.so.6...
(No debugging symbols found in /lib/x86_64-linux-gnu/libstdc++.so.6)
Reading symbols from /lib/x86_64-linux-gnu/libgcc_s.so.1...
(No debugging symbols found in /lib/x86_64-linux-gnu/libgcc_s.so.1)
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/22/ca0a83a4004122e30a69b597be96e134068616.debug...
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/63/cab9adbb271847e9c8d0baa6305bb80d6ada2e.debug...
Reading symbols from /lib/x86_64-linux-gnu/libm.so.6...
Reading symbols from /usr/lib/debug/.build-id/e6/f050696120aeb5134a14e38bc54e8ce1bdc0b5.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007b17003f7f29 in _Unwind_Find_FDE () from /lib/x86_64-linux-gnu/libgcc_s.so.1
```

Threads

Thread: Multiple independent processes sharing one address space, but have their own registers and execution stack.



info threads	Displays information about current existing threads
thread <id>	Switch execution to thread @id
break <locspec> thread <id>	Adds breakpoint which only breaks on thread @id

Thank You


A new foe has appeared!
CHALLENGER APPROACHING
GRAND ASSIGNMENT
ALL AGES OF GEEK

Advanced GDB Task

bash | breakpoints

```
gdb build/grand_task/gdb_task6

(gdb) start build/grand_task/test-dates.txt
```

Valgrind Grand Task

bash | breakpoints

```
valgrind --leak-check=full --track-origin=yes ./build/grand_task/valgrind_task6
```

Spike Grand Task

bash | breakpoints

riscv-elf-unknown-gdb spike/foo/foo

HINT

We are using **riscv64** not riscv32